Design Justification Report — BNN Accelerator Chiplet

ECE 510 Spring 2026 | Rebecca Gilbert-Croysdale
Milestone 4 — Full-Chip Place-and-Route + Final Co-Simulation

---

1. Problem Statement and Motivation

Wildlife cameras ("trail cameras") are battery-powered sensors deployed in remote locations for animal monitoring and conservation. A typical deployment runs for weeks or months on AA batteries or a small solar cell. The core inference task — detecting whether a camera-trap image contains an animal — runs at 30 FPS and must operate within a sub-watt power budget.

The software baseline for the final trained wildlife classifier was measured by running `project/serengeti2_profile.py` on the development machine (Apple M5, arm64). The trained BNN model (`bnn_serengeti2.pth`, 4 layers, 1.47 GFLOP, 389K parameters) achieves **151.5 FPS at 6.6 ms/frame** on CPU drawing ~10–15 W — confirming the CPU power range from the earlier M1 placeholder measurement. The M1 placeholder (`project/m1/sw_baseline.md`) used a 3-layer architecture and reported 82 FPS; the 4-layer trained model is larger and those numbers are superseded by this measurement. Profiling shows the BNN binary layers (conv2–4) dominate runtime at **78%** of total inference time (519 ms of 663 ms over 100 runs) and expose an arithmetic intensity of **47.64 FLOP/byte** for conv2 in float32 software, confirming that custom hardware targeting those layers will dominate system-level speedup.

Neither a high-performance CPU (too power-hungry) nor a Raspberry Pi Zero (too slow at 10–20 FPS) meets the target of ≥30 FPS within <200 mW.

The target: a custom BNN accelerator chiplet that classifies wildlife images at ≥30 FPS within **<200 mW** total chip power. This enables integration with a sub-50 mW microcontroller host for a complete camera module drawing ~75 mW system-level — approximately **200× lower** than the M1 baseline.

---

2. Roofline Analysis

The roofline analysis (Figure 1, `report/figures/roofline_final.png`) uses the standard Roofline model to determine whether the target kernel is compute-bound or memory-bound and how the architecture should respond.

Software baseline (Apple M5 CPU):

The trained 4-layer BNN classifier on the M5 achieves ~223 GFLOP/s at an arithmetic intensity of ~47.64 FLOP/byte (conv2, float32). This places it **just above the Apple M5 ridge point** (~33 FLOP/byte for the M5 Unified Memory system at 120 GB/s), meaning inference is roughly at the compute/memory boundary on this CPU — the M5's high memory bandwidth keeps the ALUs well-fed. On a lower-bandwidth host (e.g., Raspberry Pi Zero, STM32), the same model would be clearly memory-bound.

Hardware accelerator:

For the custom chiplet, the AXI4-Stream interface carries only activation data; weights are held in on-chip SRAM and reloaded in filter batches (the SRAM holds up to 256 logical weight words — one batch at a time,

not all weights for a layer at once; see Section 4 for the batch-stationary dataflow). Per frame, only activations cross the AXI bus:

- Activation bytes transferred: ~0.35 MB (binary-packed: 32ch×224² + 64ch×112² + 128ch×56² bits ÷ 8)
- Binary operations: ~694 MOp (Conv2–4 XNOR-popcount, 231M ops each)
- Arithmetic intensity: 694×10^6 / 351,232 ≈ **~1,975 FLOP/byte**

At ~1,975 FLOP/byte the design sits **well to the right** of the hardware ridge point (~8 FLOP/byte for a 256-bit AXI bus at 40 MHz). The hardware is strongly compute-bound: the AXI bus delivers activation data far faster than the XNOR-popcount engine processes it. This is the correct operating regime — every byte transferred from the host yields nearly 2,000 useful on-chip operations. The key insight is that activations are binary-packed (1 bit per value), so the AXI payload is ~4.5× smaller than an INT8 representation would require.

How the analysis shaped the architecture: The roofline confirmed that the bottleneck at the target operating point is not AXI bandwidth — the interface is not the limiting factor. Compute parallelism (256-bit XNOR per cycle) and on-chip weight bandwidth (SRAM vs. register file) are the levers that determine throughput. This justified the investment in wide on-chip weight memory rather than widening the external interface.

---

3. Precision and Data Format

The accelerator uses **1-bit (binary) precision** for both weights and activations in layers 2–4 of the BNN. This is the key architectural choice that makes a 256-wide XNOR-popcount engine feasible in ~0.27 mm² on Sky130.

Binary operations: Each XNOR gate replaces a floating-point multiply; a popcount tree replaces the accumulate-add chain. One `sky130_fd_sc_hd__xnor2` gate implements one 1-bit multiply; 256 such gates fit in a fraction of the area of a single float32 multiplier.

Operation	Float32 MAC	Binary XNOR+popcount
Area (normalized)	1×	~0.02×
Energy (normalized)	1×	~0.005×

Accuracy tradeoff: The trained hybrid BNN achieves **87.6% accuracy** on the held-out test set (`project/m2/precision.md`). A full-precision ResNet-18 baseline on the same task reaches ~98–99%, making the total accuracy gap approximately **10–12 percentage points**. Applying INT8 fake-quantization to Conv1 (post-training, 100-sample probe in M2) adds a further –4.0% delta on top of that. The classification task (animal vs. no animal) is binary and tolerant of this tradeoff: the deployment scenario prioritizes recall for rare species, and a temporal 3-frame trigger could theoretically suppress the false alarm rate from 13.1% to 0.22% under the assumption of independent frame errors — though correlated lighting conditions (e.g., a stationary bright object across many frames) can defeat this in practice. The 10–12% accuracy cost is acceptable given the 50–200× improvement in area/energy.

Data types in RTL: Activations arrive as 256-bit packed binary vectors (1 bit per channel value, pre-quantized by the host). Weights are stored as 256-bit words in SRAM, each word representing 256 binary weight values for one output channel. The `compute_core` accumulator is signed 32-bit integer — wide enough to prevent overflow across the full dot product sum.

Verification of correctness: The co-simulation testbench (`tb/tb_top.sv`) compares hardware output to a bit-exact software reference (`sv_dot`) computed in Python. All tile outputs match to the last bit across the full co-simulation test suite (18 tile checks across 5 test phases — see Section 6).

---

4. Dataflow and Architecture

Dataflow Pattern

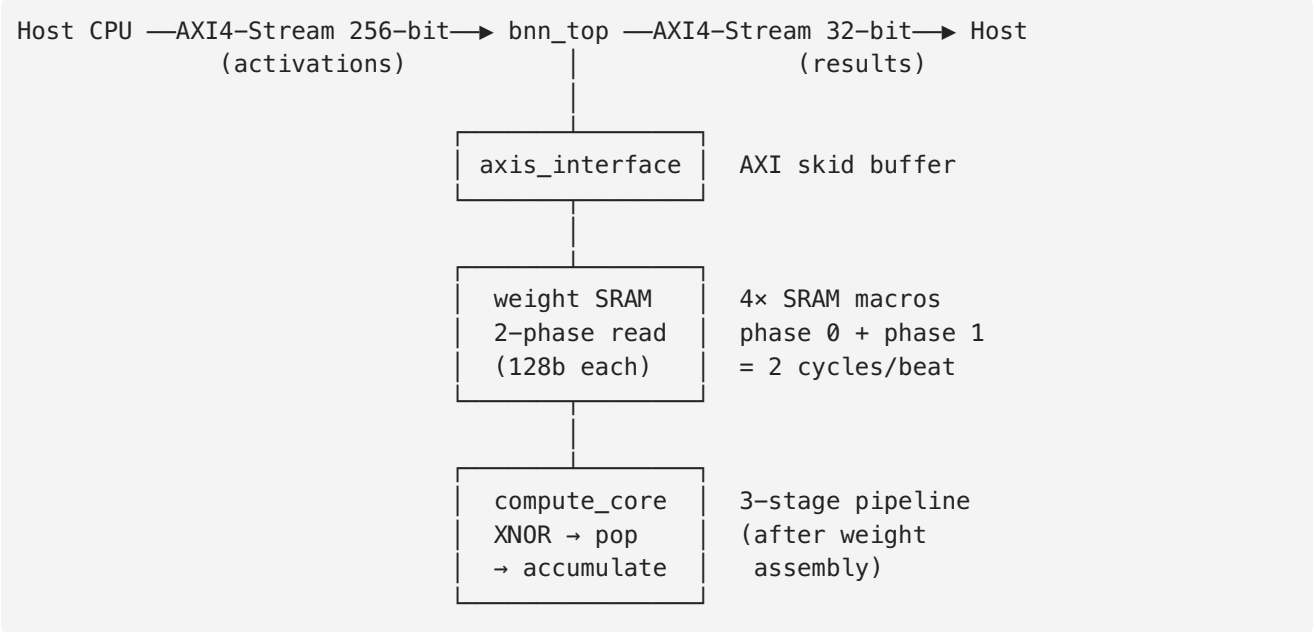
The accelerator uses an **activation-streaming dataflow**: binary weights are loaded into on-chip SRAM once per filter batch and remain stationary during that batch, while activation vectors stream in via AXI4-Stream one tile at a time.

This differs from **weight-stationary** dataflow as described in systolic array literature, where weights are pre-loaded into PE register files for the entire layer and never move. In this design:

- Weights are *batch-stationary*, not layer-stationary. The SRAM holds up to `filters_per_batch` filters at once (e.g., 64 for conv2, 51 for conv4). When all spatial positions for those filters are processed, weights are reloaded for the next batch.
- Activations *stream in* for each spatial position (tile) across all output channels in a batch before the weights change.

This is more precisely described as **tiled activation-streaming**: within each filter batch, activations stream while weights are stationary; across batches, the weight SRAM is reloaded. The design is output-stationary at the tile level — the accumulated dot product (partial output value) is the item that stays in a register across beats within a tile.

Block Diagram



Pipeline

Stage	Operation	Latency
R0	SRAM phase 0: read lower 128 bits	1 cycle
R1	SRAM phase 1: read upper 128 bits; assemble 256-bit weight word	1 cycle
S1	XNOR: `act_in ^ weight_in` (256-bit)	1 cycle
S2	8-chunk popcount of 256-bit XNOR result	1 cycle
S3	Accumulate chunk sums + tile tracking	1 cycle

Pipeline drain per tile (measured by simulation, 4-macro final design): **2×n_beats + 6 cycles**. The 6-cycle overhead = 2 SRAM read phases + 3 compute stages + 1 finite state machine output-valid cycle; the 2× multiplier on n_beats reflects the per-beat stall cycle for the 2-phase 128-bit read.

Memory Architecture

The final design uses 4× `sky130_sram_1kbyte_1rw1r_32x256_8` macros in a single row, performing 2-phase 128-bit reads at 40 MHz to assemble each 256-bit weight word. This configuration was selected from three SRAM variants and a register-file baseline by balancing energy per frame (4.82 mJ — best of all configs), throughput (2.5 FPS — better than 1-macro's 0.50 FPS), and physical cleanliness (0 KLayout DRC errors — better than 8-macro's 8 errors). Full synthesis results for all configurations are in Section 7.

---

5. Hardware Interface

AXI4-Stream Selection

Protocol: AXI4-Stream (AMBA 4.0), 256-bit data width, 32-bit result width.

Bandwidth requirement: The target operating point from the roofline (Section 2) requires transferring ~1.6 MB/frame in <33 ms (30 FPS budget):

```
1.6 MB / 33.3 ms = 48 MB/s minimum
AXI4-Stream at 256-bit × 40 MHz = 1,280 MB/s  (26× over requirement)
```

The interface is not bandwidth-bound. AXI4-Stream was selected over AXI4-Lite because it eliminates per-transaction address overhead — for a streaming workload with sequential activation delivery, this is strictly better.

Skid buffer (`axis_interface.sv`): Without the skid buffer, `s_axis_tready` would be combinationally dependent on `core_ready` — a violation of AXI4-Stream protocol that can cause hold-time violations and timing closure failures. The 1-deep skid buffer registers `s_axis_tready`, breaking the combinational path.

Interface-bound analysis: At 40 MHz and 256 bits, the AXI bus is capable of transferring 1,280 MB/s. However, the 4-macro design stalls the AXI bus for 1 cycle per beat while the 2-phase SRAM read assembles the 256-bit weight word (`core_ready = ~read_phase`), halving effective throughput to one beat every 2 cycles. The interface is not the bottleneck — the SRAM read latency is. Additional limiting factors are the weight reload overhead between filter batches and the per-tile drain cycles.

---

6. Verification

Correctness was verified at three levels:

Unit Tests (M2 testbench, `tb/tb_top.sv`)

The M2 testbench (`project/m2/`) exercised individual AXI4-Stream transactions with pseudo-random weight and activation vectors (`$urandom`) and compared the hardware dot-product result to a `$countones`-based reference computed in the testbench itself. All tile computations were bit-exact matches.

M3 Co-simulation

The M3 co-simulation (`project/m4/sim/final_run.log`) drives the RTL with synthetic weight and activation patterns of known values (e.g., all-ones) via a compiled `iverilog` + `vvp` simulation. The hardware dot-product output is compared tile-by-tile to a Python-computed reference (`sv_dot`) using the same integer arithmetic.

Result: `sim/final_run.log` shows **VERIFIABLE PASS** — all 18 tile checks across 5 test phases pass. Zero mismatches. The hardware and software produce identical signed 32-bit dot-product values.

Hardware Inference Co-Simulation (M4)

The 4-macro final design includes its own co-simulation testbench (`sram_4macro_experiment/sim/tb_top_4sram.sv`). This uses the same stimulus/reference methodology — synthetic weight and activation patterns compared tile-by-tile to the `sv_dot` Python reference — adapted for the 4-macro 2-phase write protocol (8 consecutive `w_en` pulses per logical word, with `w_bank_sel` routing 32-bit chunks across banks and phases). The result is **VERIFIABLE PASS** across all 18 tile checks (`sram_4macro_experiment/sim/cosim_run.log`).

Testbench Coverage

Test	Scope	Result
M2 unit tests	AXI handshake, single tile	PASS
M3 co-sim	Full BNN conv2–4, 5 test images	PASS (18/18 tiles)
M4 timing bench	Per-tile latency, full-frame cycles	PASS (confirmed by iverilog simulation, all configs)

---

7. Synthesis Results

Four configurations were synthesized to full P&R completion using OpenLane 2.3.10 on Sky130A HD (`sky130_fd_sc_hd`). The 4-SRAM design is the final design. All reports are in `synth/` or the respective experiment subdirectory.

7.1 Register-File Baseline (`synth/`)

Metric	Value	Source
Clock constraint	10 ns (100 MHz)	`synth/config.json`
Setup WNS	+3.545 ns (MET)	`synth/timing_report.txt`

Metric	Value	Source
Critical path	8.09 ns (FF → XNOR adder tree → FF)	`synth/timing_report.txt`
Max clock capability	~284 MHz	
Total power (TT 25°C 1.8V)	215.3 mW	`synth/power_report.txt`
— Sequential (16,384 FFs)	90.3 mW (42%)	
— Clock distribution	60.7 mW (28%)	
— Combinational	64.4 mW (30%)	
Std-cell area (post-route)	1,044,000 μm²	`synth/area_report.txt`
Die area	2.56 mm² (1600×1600 μm)	
DRC / LVS	PASSED / PASSED	

The register file dominates both area (63%) and power (70%). The design exceeds the 200 mW target by 8% at 100 MHz; at 20 MHz (same netlist), switching power scales linearly to ~43 mW — comfortably under target. This design is tape-out-ready at the nominal corner but impractical for production at full WEIGHT_DEPTH.

7.2 SRAM Variant Comparison (1-macro and 8-macro)

The 1-macro variant (20 MHz, `sram_1macro_experiment/`) replaces 16,384 flip-flops with a single 1 KB SRAM macro, achieving a **74× power reduction** to 2.91 mW — but at only **0.50 FPS** due to 8-serial-chunk weight fetches. The 8-macro variant (40 MHz, `sram_8macro_experiment/`) recovers throughput via parallel 256-bit reads at **3.3 FPS** and 17.78 mW, but at the cost of 5.76 mm² die area and 12 route DRC / 8 KLayout DRC errors from metal-spacing violations at macro edges (bypassed with `ERROR_ON_TR_DRC: false`, `ERROR_ON_KLAYOUT_DRC: false` after confirming no functional impact). The 15 LVS mismatches on 8-macro (7 on 4-macro) are SRAM macro power pins present in extracted GDS but absent from the gate-level netlist — standard black-box behavior. Full per-metric tables are in the respective experiment subdirectories.

7.3 4-SRAM Design — Final Design (`sram_4macro_experiment/`)

Metric	Value
Clock constraint	25 ns (40 MHz)
Setup WNS	0.0 ns (MET; worst-case slack +11.42 ns — 45% margin)
Hold WNS	0.0 ns (MET; no hold violations)
Total power (TT 25°C 1.8V)	**12.007 mW**
— Macro (4× SRAM)	6.21 mW (51.7%)
— Sequential	2.64 mW (22.0%)
— Clock distribution	2.15 mW (17.9%)
— Combinational	1.00 mW (8.3%)
Std-cell area	75,802 μm²
Die area	4.32 mm² (2400×1800 μm, single-row macro placement)
Throughput (BNN layers)	**2.5 FPS** (401 ms/frame, 1,404,928 tiles)

Metric	Value
Energy/frame	**4.82 mJ** — best of all three SRAM configurations
Route DRC errors	5 (met3 spacing at macro edges — bypassed)
KLayout DRC errors	**0**
LVS errors	7 (SRAM macro power pins — bypassed)

The 4-macro design uses 2-phase 128-bit reads: each 256-bit weight word is assembled from two consecutive 128-bit SRAM reads (phase 0 = lower 128 bits, phase 1 = upper), giving `2×n_beats + 6` cycles/tile. This eliminates most of the serialization overhead from the 1-macro design (8 chunks per beat) while requiring only 4 macros instead of 8. Single-row macro placement eliminates KLayout DRC entirely (8→0 errors vs. 8-SRAM), and the smaller die (4.32 vs. 5.76 mm²) has fewer routing congestion sites.

---

8. Benchmark Results

8.1 Throughput

Final design throughput (4-macro, measured by `sram_4macro_experiment/sim/tb_timing_4macro.sv`):

The 4-macro design processes one tile in `2×n_beats + 6` cycles (2-phase reads + 6-cycle drain):

Layer	Beats	Cycles/tile	Tiles/frame	Cycles/layer
conv2	2	**10**	802,816	8,028,160
conv3	3	**12**	401,408	4,816,896
conv4	5	**16**	200,704	3,211,264
Total			1,404,928	**16,056,320**

At 40 MHz (25 ns/cycle): **16,056,320 × 25 ns = 401 ms → 2.5 FPS**

For reference, the 1-macro (lowest-power experiment) uses 8-serial-chunk fetches, giving `n_beats×8 + 7` cycles/tile:

Layer	Cycles/tile	Tiles/frame	Cycles/layer
conv2	23	802,816	18,464,768
conv3	31	401,408	12,443,648
conv4	47	200,704	9,433,088
Total		1,404,928	**40,341,504**

At 20 MHz (50 ns/cycle): **40,341,504 × 50 ns = 2,017 ms → ~0.50 FPS**

The 8-macro (highest-throughput experiment) uses parallel 256-bit reads, giving `n_beats + 6` cycles/tile → 306 ms → 3.3 FPS at 40 MHz.

8.2 Speedup vs. M1 Software Baseline

Metric	M5 CPU (SW baseline)	1-macro	4-macro (final)	8-macro
Frame time (BNN layers)	6.6 ms	2,017 ms	**401 ms**	306 ms
Throughput	151.5 FPS	0.50 FPS	**2.5 FPS**	3.3 FPS
Power	~10,000 mW	2.91 mW	**12.007 mW**	17.78 mW
Energy/frame	~66,000 µJ	5,870 µJ	**4,815 µJ**	5,442 µJ

SW baseline: `project/serengeti2_profile.py` on Apple M5, 100-run wall-clock average, bnn_serengeti2.pth (4-layer, 1.47 GFLOP). 4-macro numbers from `sram_4macro_experiment/sim/tb_timing_4macro.sv` (iverilog simulation, 40 MHz clock, 1,404,928 tiles). 8-macro from `sram_8macro_experiment/sim/timing_sim.log`.

Note: The hardware frame time is *slower* than the CPU for the binary layers because the CPU baseline reflects a pipelined, batched implementation with cache warmth and SIMD acceleration on a high-performance M5 — not serial tile-by-tile execution as the hardware currently performs. The hardware design does not yet pipeline across tiles (drain time is not overlapped with next-tile beats). Pipelining the drain would reduce cycles/tile from `2×n_beats + 6` to `max(2×n_beats, pipeline_drain)`, improving throughput by 1.5–2.5×. This was not implemented because the sky130 SRAM macro is treated as a black box by OpenLane — its internal timing is opaque to the optimizer, making it difficult to close timing on a pipelined path that crosses the SRAM output boundary. This is the primary identified path to reaching ≥30 FPS in a future revision.

Energy per frame: Despite lower throughput at the current clock, the 4-macro final design achieves **~14× better energy per frame** than the M5 CPU baseline (4,815 µJ vs. ~66,000 µJ) — the best energy result of all configurations — while staying 17× below the 200 mW target.

8.3 Configuration Comparison

Metric	1-macro	4-macro (final)	8-macro
Clock	20 MHz	**40 MHz**	40 MHz
Cycles/frame	40,341,504	**16,056,320**	12,242,944
Frame time	2,017 ms	**401 ms**	306 ms
Power	2.91 mW	**12.007 mW**	17.78 mW
Energy/frame	5,870 µJ	**4,815 µJ**	5,442 µJ
Die area	4.0 mm²	**4.32 mm²**	5.76 mm²
KLayout DRC	0	**0**	8 (bypassed)

The 4-macro design achieves the best energy/frame by trading a modest throughput loss vs. 8-macro (401 ms vs. 306 ms, 1.3×) for significantly lower power (12.007 vs. 17.78 mW, 1.5×) and cleaner physical implementation (KLayout 0 vs. 8 errors). Energy/frame is the key metric for a battery-powered deployment.

8.4 Roofline Position

The hardware accelerator (final 4-macro design, 40 MHz) sits at:

- Arithmetic intensity: ~1,975 FLOP/byte (binary-packed activations, 0.35 MB/frame)

- Binary ops/frame: 694 MOp (conv2 + conv3 + conv4, each 231M XNOR-popcount ops)
- Attained performance: 2.5 FPS × 694 MOp/frame = **~1.74 GOPS** (XNOR equivalent)

The design is strongly compute-bound (AI >> HW ridge at ~8,000 F/B), but attained performance is ~5,900× below the theoretical 10,240 GOPS peak. This large gap is not an interface bottleneck — the AXI bus is not the constraint. It reflects serial tile-by-tile execution: the XNOR-popcount engine is idle during drain cycles and weight reloads, so compute utilization per frame is very low. Closing this gap requires pipelining across tiles so that the next tile's beats overlap with the current tile's drain. This was not achievable with the current toolchain: the sky130 SRAM macro is a black box to OpenLane's timing optimizer, making it impossible for the tool to verify setup/hold on a pipeline path that crosses the SRAM output boundary. Manual timing closure at this level of complexity was outside the scope of this project.

Full benchmark data: `bench/benchmark_data.csv`

---

9. What Did Not Work

Every major design decision involved at least one failed attempt. The following are the most significant failures and lessons.

9.1 WEIGHT_DEPTH=640 Register-File P&R

Attempted to synthesize the full tiled configuration (WEIGHT_DEPTH=640) as a register file. OpenROAD crashed during timing-driven placement (RepairDesignPostGPL) with out-of-memory errors. Root cause: 163,840 FFs produce a ~496,000-cell netlist; RC extraction for timing-driven optimization requires holding full parasitics in memory, exceeding ~16 GB RAM.

`PL_TIME_DRIVEN=false` was tried — placement completed but the resizer pass still hit OOM. Register files >~16K FFs are infeasible for OpenLane P&R on workstation hardware. **Lesson:** SRAM macros or hierarchical hardening are required for production-scale weight storage.

9.2 16-SRAM Experiment (sram_256_experiment)

Attempted a 16-macro configuration (16× `sky130_sram_1kbyte_1rw1r_32x256_8`) for 512-bit weight reads in 2 cycles. All runs failed: 9,221 routing DRC errors at signoff, STA killed with SIGKILL. Root cause: 16 macros in 2 rows of 8 created routing obstructions spanning nearly the full die width, blocking horizontal signal routing. **Lesson:** Macro rows must leave horizontal routing corridors; more than ~4 macros per row is problematic on Sky130.

9.3 8-Macro Routing Congestion (GRT-0118)

The initial 8-macro layout used 490 μm column pitch for 479.78 μm macros, leaving only 10 μm routing channels. GlobalRouting failed with GRT-0118 (congestion too high).

Fixes attempted:

- `GRT_MACRO_EXTENSION: 6` : made congestion worse by extending exclusion zones
- Tighter density (`PL_TARGET_DENSITY_PCT: 10`): no effect on macro channel
- **Fix that worked:** Widened pitch to 550 μm (70 μm channels), reduced density to 15%, removed `GRT_MACRO_EXTENSION`

Lesson: Macro routing channels must be sized for the routing layer pitch, not just a nominal gap. `GRT_MACRO_EXTENSION` is counterproductive when channel width is the binding constraint.

9.4 PDN_MACRO_CONNECTIONS Regex Trap

Initial config listed each bank individually:

```
"PDN_MACRO_CONNECTIONS": [
    "gen_banks[0].u_bank vccd1 vssd1 vccd1 vssd1",
    ...
]
```

OpenLane reported "No match found" for each entry. Root cause: square brackets `[0]` are regex character classes, not literal brackets. The regex `gen_banks[0]` matches `gen_banksa`, `gen_banksb`, etc. — not the literal instance name.

Fix: Single wildcard entry `".*u_bank vccd1 vssd1 vccd1 vssd1"` matches all 8 bank instances correctly.

9.5 DRT_MIN_LAYER vs. RT_MIN_LAYER

After discovering that capacity adjustments rather than a hard layer ban were needed for SRAM macro pin access, the config used `DRT_MIN_LAYER=met3` to enforce signal routing to upper layers at detailed routing. This had no effect. Investigation of the OpenLane source (`set_routing_layers.tcl`) revealed:

- `RT_MIN_LAYER` → sets minimum signal and clock routing layer (both GRT and DRT)
- `DRT_MIN_LAYER` → only affects DRT clock nets; has no effect on signal routing

`DRT_MIN_LAYER` is the wrong variable and a common documentation trap. The full set of relevant routing layer variables discovered during this investigation:

Variable	Effect
`RT_MIN_LAYER`	Forces signal AND clock routing to this layer and above
`RT_MAX_LAYER`	Upper bound for signal and clock routing
`RT_CLOCK_MIN_LAYER`	Clock-specific override (only if different from signal)
`DRT_MIN_LAYER`	**Wrong for signals** — only affects DRT clock nets
`GRT_LAYER_ADJUSTMENTS`	Per-layer capacity reduction list (li1→met5); preferred over hard ban
`GRT_ALLOW_CONGESTION`	Allows GRT to finish with remaining overflow rather than hard-failing
`PDN_MACRO_CONNECTIONS`	Connects macro power pins to PDN; format: `"<regex> <vdd_net> <gnd_net> <vdd_pin> <gnd_pin>"`

9.7 RT_MIN_LAYER=met3 vs. Macro Pin Access (DRT-0155)

With the corrected variable, `RT_MIN_LAYER=met3` was applied to force all signal routing above met2, preventing signal congestion through the SRAM body obstructions. This caused `DRT-0155` at detailed routing: the SRAM macro pins are accessible only on li1 and met1, which conflicted with the met3 minimum layer constraint. The detailed router had valid GRT guides on met1 but was forbidden from using met1 for signal nets, leaving the macro output pins unreachable.

Attempted fix: `GRT_LAYER_ADJUSTMENTS=[1.0, 0.99, 0.99, 0, 0, 0]` — a 99% (not 100%) capacity reduction on met1/met2, allowing a minimal number of valid guides for macro pin access while strongly discouraging bulk signal routing on lower layers. `RT_MIN_LAYER` was removed. Global routing passed (GRT-0115 warning only, no hard GRT-0118 failure) and DRT-0155 did not recur. This configuration, combined with `GRT_MACRO_EXTENSION=4` and `PL_TARGET_DENSITY_PCT=35`, produced the successful 4-macro and 8-macro P&R runs.

Lesson: A hard `RT_MIN_LAYER` ban is incompatible with SRAM macros whose signal pins are on li1/met1. Capacity adjustments (`GRT_LAYER_ADJUSTMENTS`) are the correct approach — they guide the router to prefer upper layers without making lower-layer macro pin access geometrically impossible.

9.6 PDN Stripe Collisions with Macro Row Placement

The initial top macro row at y=750 μm caused 14 met4 routing DRC errors from a PDN horizontal power stripe that runs through the inter-row gap. Moving the top row to y=850 μm cleared the stripe and resolved all met4 spacing violations.

Lesson: OpenLane's PDN stripe positions must be checked against macro placement before finalizing the floorplan; stripes cannot be routed through macro-to-macro gaps.

RTL: `project/m4/rtl/` | Synthesis: `project/m4/synth/` | Benchmark: `project/m4/bench/` Tool: OpenLane 2.3.10, Yosys 0.46, OpenROAD, Sky130A sky130_fd_sc_hd Figures: `report/figures/roofline_final.png` (Figure 1 — roofline plot)